SEMT20003 Practice Paper 2

Classification, Backpropagation, and Overfitting

Time: 3 hours **Total Marks:** 100

Part I: Multiple Choice Questions (25 marks)

Answer all questions. Each question has 4 possible options. Between ONE and THREE of these options may be correct. For full marks (5 marks per question), you must state exactly the correct options. No partial credit is awarded.

1. **When training a classification model using Gradient Descent, why is the Sigmoid function preferred over the Heaviside step function for generating probabilities?**

   A) The Heaviside function has a gradient of exactly zero almost everywhere, making learning impossible.

   B) The Sigmoid function perfectly avoids the vanishing gradient problem in deep networks.

   C) The Sigmoid function provides a smooth, differentiable landscape everywhere.

   D) The Heaviside function outputs values outside the range $[0, 1]$.

2. **Which of the following functions contain points that are mathematically non-differentiable?**

   A) Mean Squared Error (MSE) loss

   B) Mean Absolute Error (MAE) loss

   C) Rectified Linear Unit (ReLU) activation

   D) Sigmoid activation

3. **Which of the following statements correctly describe the mechanics of backpropagation through a computational graph?**

   A) Gradients are computed starting from the input nodes and moving forward to the output node.

   B) The chain rule is applied by multiplying the local gradient at a node by the upstream gradient received from its output.

   C) An addition node routes the exact same upstream gradient equally to all of its input branches.

D) A multiplication node $f = x \times y$ uses the value of y as the local gradient for x, and x for y.

4. **When observing loss curves during the training of a neural network, which of the following combinations strongly indicates that the model is overfitting?**

   A) The training loss steadily decreases toward zero.

   B) The validation loss initially decreases but then begins to steadily increase.

   C) Both the training and validation loss fail to decrease from their initial values.

   D) The training loss increases while the validation loss decreases.

5. **How does Early Stopping act as a regularization mechanism?**

   A) It periodically evaluates the model on a held-out validation set during training.

   B) It halts training immediately when the training loss reaches zero.

   C) It stops training when validation performance ceases to improve, typically restoring the best previous weights.

   D) It automatically decreases the learning rate when the training loss plateaus.

Part II: Programming-related Questions (35 marks)

Question 1: Dropout Implementation (15 marks)
Write a Python function using NumPy that implements the Dropout regularization technique for a given input tensor (array) `x`.

Your function `dropout(x, p, is_train)` should accept:

- `x`: A NumPy array of any shape containing intermediate activations.
- `p`: A float between 0.0 and 1.0 representing the probability of *dropping* a neuron (setting it to zero).
- `is_train`: A boolean indicating whether the network is in training mode (`True`) or evaluation mode (`False`).

The function must return the modified tensor. Ensure you implement *inverted dropout*, meaning the active neurons are properly scaled during the training phase so that the expected value remains constant, and no scaling is required during the evaluation phase.

Question 2: Backward Pass for Dense + Sigmoid Layer (20 marks)
You are given the intermediate variables of a forward pass for a single dense layer followed by a Sigmoid activation:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$$
$$\mathbf{A} = \sigma(\mathbf{Z})$$

Write a Python function `backward_pass(X, W, A, d_out)` that calculates the gradients of the loss with respect to `X`, `W`, and `b`.

The function accepts:

- `X`: Input array of shape `(N, D_in)`
- `W`: Weight array of shape `(D_in, D_out)`
- `A`: The post-activation output array of shape `(N, D_out)` (i.e., the result of the Sigmoid).
- `d_out`: The upstream gradient of the loss with respect to `A`, of shape `(N, D_out)`.

Return a tuple containing `(dX, dW, db)`.

Part III: Analytic Questions (40 marks)

Question 1: Computational Graph Tracing (20 marks)
Consider a computational graph defining a function $f(x, y)$:

$$
\begin{aligned}
p &= x \cdot y \\
q &= \max(0, p) \\
r &= x - y \\
s &= r^2 \\
f &= q + s
\end{aligned}
$$

Given the inputs $x = 2$ and $y = -1$:

- **a) (5 marks):** Calculate the forward pass values for nodes p, q, r, s, and f.
- **b) (15 marks):** Analytically trace the backpropagation algorithm through this graph to compute the exact numerical values for the gradients $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$. Show the local gradient calculations at each node.

Question 2: Weight Decay vs. Decoupled Weight Decay (20 marks)
Standard L2 Regularization (Weight Decay) is often added directly to the loss function: $L' = L + \frac{\lambda}{2}\|\mathbf{w}\|^2$.

- **a) (10 marks):** Derive the gradient of the modified loss L' with respect to the weights $\mathbf{w}$. Show how this changes the standard Stochastic Gradient Descent (SGD) update rule.
- **b) (10 marks):** Advanced optimizers like Adam use moving averages of gradients and squared gradients. Explain mathematically why adding L2 regularization to the loss function (standard Weight Decay) behaves undesirably when combined with Adam. Describe how *Decoupled Weight Decay* (as used in AdamW) solves this issue by modifying the update rule directly rather than the loss function.

Mark Scheme: Practice Paper 2

Part I: Multiple Choice Questions (25 marks)

5 marks per question. Must state exactly the correct combinations. Partial credit is 0.

1. **A, C**
   Rationale: A is true; the Heaviside function is flat everywhere except at $x = 0$, meaning its derivative is 0, providing no gradient signal to update weights. C is true; Sigmoid is differentiable for all real numbers. B is false; Sigmoid actually *suffers* from the vanishing gradient problem because its gradients approach zero for very large or small inputs. D is false; Heaviside outputs 0 or 1, which are perfectly valid probabilities.

2. **B, C**
   Rationale: MSE (A) and Sigmoid (D) are smooth continuous functions and differentiable everywhere. MAE (B) has a sharp kink at exactly 0 error. ReLU (C) has a kink at exactly $x = 0$, making them non-differentiable at those specific points.

3. **B, C, D**
   Rationale: A is false; backpropagation computes gradients starting from the output and moving backward to the inputs. B is the exact definition of the chain rule in this context. C is true; addition distributes the gradient. D is true; the local gradient of $x \cdot y$ with respect to x is y, and vice versa.

4. **A, B**
   Rationale: Overfitting occurs when the model memorizes the training data but fails to generalize. This is visualized by the training loss continuing to decrease (A) while the validation loss begins to increase (B).

5. **A, C**
   Rationale: Early stopping involves evaluating on a separate validation set (A) and halting the training loop when validation metrics worsen over a certain number of epochs, preventing the model from fitting the noise in the training set (C). B and D describe different mechanisms entirely.

Part II: Programming-related Questions (35 marks)

Question 1 (15 marks)

- **3 marks:** Correctly identifying return behavior for evaluation mode.
- **5 marks:** Correctly generating a boolean mask based on probability p (or $1 - p$).
- **5 marks:** Correctly applying the inverted dropout scaling factor $\frac{1}{1-p}$.
- **2 marks:** Correctly applying the mask to the input tensor `x`.

```python
import numpy as np

def dropout(x, p, is_train):
    if not is_train:
        # 3 marks: Return unmodified tensor during evaluation
        return x

    # 5 marks: Create random mask. We keep neurons with probability (1-p)
    keep_prob = 1.0 - p
    mask = (np.random.rand(*x.shape) < keep_prob)

    # 7 marks: Apply mask and scale (Inverted Dropout)
    out = (x * mask) / keep_prob
```

```

    return out
```

Question 2 (20 marks)

- **6 marks:** Correctly computing the local gradient of the sigmoid $\sigma(z)(1-\sigma(z))$. Since we have `A`, this is `A * (1 - A)`.
- **4 marks:** Applying the chain rule to get `dZ = d_out * local_sigmoid_grad`.
- **4 marks:** Computing `dW` via matrix multiplication (`X.T @ dZ`).
- **3 marks:** Computing `db` by summing `dZ` over the batch dimension.
- **3 marks:** Computing `dX` via matrix multiplication (`dZ @ W.T`).

```python
import numpy as np

def backward_pass(X, W, A, d_out):
    # Local gradient of sigmoid: A * (1 - A)
    # Upstream gradient through sigmoid (chain rule)
    dZ = d_out * A * (1.0 - A)

    # Gradients for weights, bias, and input
    dW = np.dot(X.T, dZ)
    db = np.sum(dZ, axis=0)
    dX = np.dot(dZ, W.T)

    return dX, dW, db
```

Part III: Analytic Questions (40 marks)

Question 1: Computational Graph Tracing (20 marks)

Part a) Forward Pass (5 marks)

- $p = x \cdot y = 2 \cdot (-1) = -2$
- $q = \max(0, -2) = 0$
- $r = x - y = 2 - (-1) = 3$
- $s = r^2 = 3^2 = 9$
- $f = q + s = 0 + 9 = 9$

(1 mark for each correct node value)

Part b) Backward Pass (15 marks)

- $\frac{\partial f}{\partial f} = 1$
- $\frac{\partial f}{\partial q} = 1$ and $\frac{\partial f}{\partial s} = 1$ (Addition node routes 1.0 upstream) *(2 marks)*
- Branch 1 (through s):
  - $s = r^2 \implies \frac{\partial s}{\partial r} = 2r = 6$
  - $\frac{\partial f}{\partial r} = \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial r} = 1 \cdot 6 = 6$ *(3 marks)*
  - $r = x - y \implies \frac{\partial r}{\partial x} = 1$ and $\frac{\partial r}{\partial y} = -1$
  - $\left(\frac{\partial f}{\partial x}\right)_r = 6 \cdot 1 = 6$ and $\left(\frac{\partial f}{\partial y}\right)_r = 6 \cdot (-1) = -6$ *(3 marks)*

- Branch 2 (through q):
  - $q = \max(0, p)$. Since $p = -2 < 0$, the local gradient $\frac{\partial q}{\partial p} = 0$.
  - $\frac{\partial f}{\partial p} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial p} = 1 \cdot 0 = 0$ *(3 marks)*
  - Because the gradient stops at the ReLU node, $\left(\frac{\partial f}{\partial x}\right)_q = 0$ and $\left(\frac{\partial f}{\partial y}\right)_q = 0$. *(2 marks)*
- Final summation of gradients:
  - $\frac{\partial f}{\partial x} = 6 + 0 = \mathbf{6}$
  - $\frac{\partial f}{\partial y} = -6 + 0 = \mathbf{-6}$ *(2 marks)*

Question 2: Weight Decay vs. Decoupled Weight Decay (20 marks)

Part a) Standard L2 Regularization (10 marks)

- **4 marks:** Differentiating L'. $\frac{\partial L'}{\partial \mathbf{w}} = \frac{\partial L}{\partial \mathbf{w}} + \lambda \mathbf{w}$.
- **6 marks:** Showing the SGD update.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\partial L'}{\partial \mathbf{w}}$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \left(\frac{\partial L}{\partial \mathbf{w}} + \lambda \mathbf{w}_t \right)$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t (1 - \eta\lambda) - \eta \frac{\partial L}{\partial \mathbf{w}}$$

  This shows why it is called "weight decay": the weight is exponentially decayed by a factor of $(1 - \eta\lambda)$ at each step.

Part b) Decoupled Weight Decay in Adam (10 marks)

- **5 marks:** Explaining Adam's interference. In standard weight decay, the regularization gradient $\lambda \mathbf{w}$ is added to the loss gradient *before* calculating Adam's moving averages. Adam divides updates by the square root of the moving average of squared gradients. If a weight has a large gradient from the data loss, the denominator becomes large, scaling down the *entire* update step, including the $\lambda \mathbf{w}$ penalty. Thus, parameters with large gradients are penalized less than they should be.
- **5 marks:** Explaining the Decoupled solution. Decoupled weight decay (AdamW) does not add $\lambda \mathbf{w}$ to the loss gradient. Instead, it computes the Adam update step purely based on the data loss $\frac{\partial L}{\partial \mathbf{w}}$, and then explicitly subtracts $\eta\lambda \mathbf{w}_t$ from the weights at the very end. This ensures the weight decay rate is completely independent of the dynamic learning rates computed by Adam.